EXPERIMENT 08

K-Means Clustering – Optimal Clusters (Cell by Cell)

Aim: To determine the optimal number of clusters in the Iris dataset using the Elbow and Silhouette methods, apply K-Means clustering, and visualize the clusters using PCA.

Dataset: Built-in Iris dataset from scikit-learn.

Cell 1: Fix warnings and import os

```
import os
os.environ["OMP_NUM_THREADS"] = "1"

import warnings
warnings.filterwarnings("ignore")
```

Cell 2: Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

Cell 3: Load Iris dataset

```
data = load_iris()

X = data.data
y = data.target
```

Cell 4: Convert into DataFrame

```
df = pd.DataFrame(X, columns=data.feature_names)

print("First Five Records:")
print(df.head())
```

Cell 5: Dataset Information

```
print("\nDataset Information:")
df.info()
```

Cell 6: Check Missing Values

```
print("\nMissing Values:")
print(df.isnull().sum())

print("\nDataset Shape:", df.shape)
```

Cell 7: Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Cell 8: Elbow Method

```
wcss = []

for k in range(1, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    wcss.append(model.inertia_)

# Elbow Graph
plt.figure(figsize=(7, 5))
plt.plot(range(1, 11), wcss, marker='o')

plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
```

```
plt.xticks(range(1, 11))
plt.grid()
plt.show()
```

Cell 9: Silhouette Method

```
silhouette_scores = []

for k in range(2, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

    print("K =", k, "Score =", round(score, 4))
```

Cell 10: Find Optimal K

```
optimal_k = list(range(2, 11))[
    np.argmax(silhouette_scores)
]

print("\nOptimal Number of Clusters:", optimal_k)

# Silhouette Graph
plt.figure(figsize=(7, 5))
plt.plot(range(2, 11), silhouette_scores, marker='o')

plt.title("Silhouette Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.xticks(range(2, 11))
plt.grid()
plt.show()
```

Cell 11: Final K-Means Model

```
kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)

# Add clusters to DataFrame
df['Cluster'] = clusters

print("\nFinal Cluster Results:")
print(df.head(10))
```

Cell 12: Cluster Distribution

```
print("\nCluster Distribution:")
print(df['Cluster'].value_counts().sort_index())
```

Cell 13: Final Silhouette Score

```
final_score = silhouette_score(X_scaled, clusters)

print("\nFinal Silhouette Score:", round(final_score, 4))
```

Cell 14: PCA Visualization

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
centers_pca = pca.transform(kmeans.cluster_centers_)

plt.figure(figsize=(8, 5))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=clusters,
    cmap='viridis',
    s=50
)

plt.scatter(
```

```
    centers_pca[:, 0],
    centers_pca[:, 1],
    color='black',
    marker='X',
    s=150,
    label='Centroids'
)

plt.title("K-Means Clustering on Iris Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
```

Cell 15: Conclusion

```
print("\nConclusion:")
print("K-Means Clustering completed successfully.")
print("Optimal K:", optimal_k)
print("Final Silhouette Score:", round(final_score, 4))
```

Note: Run the cells in order, from Cell 1 to Cell 15. The optimal K, cluster distribution, and Silhouette Score are calculated when the code is executed.